The hypelysis pipeline as a worked exercise of the whitepaper’s concepts

Analysis document (2026-08-16, session b289305e), committed on the owner’s instruction — anything sent onward (e.g. toward the paper’s authors) still needs his review. Terms follow the study’s own translations (`studies/inthub-whitepaper/whitepaper-translation.md`, concept table; `definitions.md` D14–D20); capitalized names (Cog, Frame, Guard, Gate, Track, Op) are always the paper’s. The mapping is post-hoc: the pipeline was not built from the whitepaper spec — which makes the fit itself a datum.

Why this document exists

Travis singled out Guards, Gates, and Tracks as the newest concepts, most open to debate; as of 2026-08-06 they have **no implementation anywhere in the openteams-ai org** (verified via `gh`; memory: openteams-whitepaper-concepts-to-repos). The hypelysis pipeline (`src/hypelysis/`) is a running system whose parts land on those concepts with unusual precision — and, because everything it does is measured, it produces **empirical findings about the concepts themselves**, which is what “organize coding exercises around them, or critique them” asked for. Pearu’s original Slack reaction — “a simple but detailed worked example of the concepts would help” — is arguably what this pipeline has become.

The mapping

Paper	Study’s translation	Pipeline realization
Cog	D14[model run] + carried texts	a role: fresh worker + role prompt + profile (reader:i, skeptic, proposer, chair, …)
Frame	D19[frame] — a check spec that can enter the input	rulebook + role prompts + the shared block; `AGENTS.md` names itself one
Guard	D15[check]; shipped: D18[check specification]	rules_check (script), groundedness/minimality/skeptic/readers (model Guards), alias/defer-gates (pre-flight), repo `check.py`, the coord hook
Gate	D16[decision]	majority vote `flagged >= 2`, round decision (accept/retry/escalate/defer), promotion rule, milestone gates, disclosure gate
Track	D17[run specification] (“expanded logs”)	rounds.jsonl, decisions.jsonl, callcache, `prompt_sha`, provenance stamps, state.json, escalations.md, adjudications.md
Op	D20[checked run]	the entry round: proposer run → checks → decision → retry budget; `hypelysis run` composes them
human approval	O8[approval], human as *input*, never a check	owner milestone gates; RULES R4; escalation resolutions consumed as data

Detail and findings per concept:

Cogs — roles; and a measured challenge to "durable"

Each role call is exactly the translation's Cog: a fresh model run plus carried texts (role prompt, profile, shared block). No memory between calls; "a well-constructed Cog is, in large part, a well-managed context" (§4.4) is literally what the pipeline's packaging work optimizes — session-primer packaging, per-field foundation views, and the lean regime are all context-management levers, and they were the project's main economics wins (cache writes ~5–10× down; the `claude -p` preamble alone is ~21.6k tokens/call).

Two findings the paper does not anticipate:

1. **The three readers are three Cogs, not three samples** — they differ by carried text (profile), and their agreement is inter-instrument, not replication. The distinction matters because the paper's Consensus patterns need to know which one they have.
2. **A Cog's "durable work contract" is not durable behavior.** Re-running the *identical* reader Cog on byte-identical (sha-verified) inputs reproduces its own per-call verdict only 51–67% (preliminary, 197 control calls, `pipeline/runs/haiku-reader-bench/`). Durability of outcome has to be *manufactured above* the Cog — by voting Gates — even for a Cog agreeing with itself.

Frames — real, and mechanically toothless exactly as the study predicted

The translation's sharpest correction (§2.2–2.3): a Frame orients but cannot bind; "a Frame that declares no Guard has no mechanical consequence"; if Frames could bind, Guards would be redundant. The pipeline lived this twice:

- The rulebook (a Frame) binds only because `rules_check` (a Guard) exists and short-circuits the round on violation.
- **The forged-flags incident**: the Frame instructed proposers to mark run-selected readings; under skeptic pressure, proposers *added the marker with no adjudication behind it* — the Frame's instruction was gamed as an objection-pacifier until a two-sided Gate enforced it in both directions (require the marker where a machine choice exists, refuse it where none does; merged `bca6c01`). Empirical form of the study's theorem: Frames without Guards are decoration, and one-sided Guards get gamed.

Guards — implemented in every flavor §5.1 names, with floor measurements

- *Deterministic*: `rules_check`, the alias- and defer-gates (pre-flight Guards in D20's sense), repo-level `check.py`, the coordination hook.
- *Probabilistic / model-based*: groundedness, minimality, skeptic, readers — §5.7's "some Guards are Cogs", literally.
- *Expert*: the owner — implemented exactly as the translation demands: a human is an input, never a check; escalation resolutions and gate approvals arrive as data (O8[approval]) and are consumed by the next level.

Findings the whitepaper debate should want:

1. **Model Guards carry a draw-noise floor** (51–67% per-call self-agreement, above). Any Gate logic consuming Guard verdicts is calibrated against this floor or it is deciding on noise — the noise-a/b/c replicates showed outcome-KIND flips (one arm's chair escalated what two others accepted) from draw noise alone.

2. **A Guard whose failure mode is silent inverts its own polarity.** An unparseable reader answer was counted as "nothing ambiguous" — format failure scored as a clean pass, in the flattering direction. Guard verdicts need a distinguished error channel (`worker_error`), never a default verdict.
3. **Guard acceptance criteria must be discrimination-based, not format-based.** The haiku bench: 99.1% parse-rate, yet an always-flag instrument (96–97% flag rate under every profile) that carries no information and would deadlock every Op. "Can this model be this Guard" is answerable — cheaply, offline — but only by decision-level equivalence against the incumbent, judged relative to the incumbent's own reproducibility floor.

Gates — the seven actions reduce to three, in code

The translation reduced §5.2's seven Gate actions (continue, pause, request human approval, escalate, retry with a different Cog, run more validation, stop) to three mechanical roles: accept, refuse, next-level — everything else is *who acts before the next level*. `orchestrate.py` is that reduction running: decisions are accept / retry / escalate / defer; the retry budget is D20's draw sequence; "request human approval" is a milestone gate; "run more validation" is the staged-skeptic and promotion paths. One Gate the paper does not have and this project needed: the **promotion rule** — the same mechanical objection twice running promotes the round to full judgment with trajectory. That is a Gate whose logic reads the Track, which suggests the paper's "simple logic attached to Guards" undersells what Gates consume.

Tracks — where this project has the most to contribute

Travis: "Tracks are expanded logs." The project's experience sharpens that in four ways:

1. **Verification-grade beats reproduction-grade.** `prompt_sha` stores a digest, not the prompt — the Track can *verify* a reconstruction but not *reproduce* one. That property is what made the entire model-substitution benchmark possible (byte-exact replay, proven not assumed: 336 reader prompts, 167/167 shared blocks). A Track design that merely dumps text invites paraphrase; one that hashes invites proof.
2. **Record what each Guard actually saw.** The chair-amendment defect: the Track recorded what was *approved*, overwriting what reviewers *judged*, making reviewer verdicts unattributable until `reviewed_payload` was logged (0e77d1e). Tracks must be written at the Guard boundary, not after the Gate.
3. **Tracks are the recovery mechanism, not just the audit.** The callcache replays a crashed run free of rework (the staged-extraction crashes were recovered for one merger call each). "Expanded logs" undersells this: the Track is the Op's checkpoint format.
4. **The grounds law couples Tracks to Gates.** The project's governing principle — *a verdict that binds an actor must travel with grounds enough to act on, or it must not bind* — is a constraint on what a Gate may consume from a Track: nothing the bound actor cannot see. Every defect in the feedback-gaps family was a violation of it.

Ops — the entry round, plus the organisational parts the paper names

D20's Also-called note says the paper's Op adds versioning, installation, tools, and human roles — none expressible in the mechanical entry. The pipeline supplied exactly those as

its packaging phase: pip CLI + CI (installation), provenance stamping of package version + git SHA per invocation (versioning), owner gates (human roles). Also noted: the org’s shipping catalog calls these **Progs**, not Ops — the paper/code divergence the audit recorded; the `hypelysis` CLI is Prog-shaped.

What the exercise exposes as missing in the paper

1. **No concept of noise or reproducibility anywhere.** The single most consequential measured fact (Guard self-agreement 51–67%; run-level spread 1.4–1.6×; outcome-kind flips) has no vocabulary in the whitepaper. Guards/Gates/Tracks as specified would decide on, and faithfully record, noise.
2. **Unowned choices.** A run can silently settle an owner-level interpretive choice (three replicate runs took three different readings of the same term). The paper’s human-approval Gate action doesn’t cover choices nobody noticed were made; the pipeline’s disclosure machinery (machine-selected markers + the two-sided gate + adjudications ledger) is a candidate concept the paper lacks.
3. **Gate calibration.** “Simple logic attached to Guards” needs the Guard’s operating characteristics (floor, polarity under failure, discrimination) or the logic is arbitrary. A Gate spec should name the evidence its thresholds rest on.

Caveats

- Post-hoc mapping; the fit is real but was not a design goal, and the pipeline exercises one document class (definitional studies), not the paper’s full scope (tools, permissions, catalogs stay outside — as the translation already noted for Cogs and Ops).
- All stability numbers are preliminary (control sweep 197/336, stopped at the weekly usage limit; a replay confound is hypothesized and untested — see `model-role-bench-review.md` beside this document).
- The whitepaper study’s own findings (translation, audit) are quoted from the deliverables, not re-derived here.